Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Read the Titanic dataset and remove rows without an age.
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/dat
df.dropna(subset=['Age'], inplace=True)

# Use age and ticket class to predict fare.
features = ['Age', 'Pclass']
X_reg = df[features]
y_reg = df['Fare']
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
print(f'Regression: {len(X_train_reg)} training rows, {len(X_test_reg)}
```

```
Regression: 571 training rows, 143 test rows
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# Train a linear regression model on the training data.
lr_model = LinearRegression()
lr_model.fit(X_train_reg, y_train_reg)

# Predict fares for passengers the model has not trained on.
y_pred_reg = lr_model.predict(X_test_reg)

# Measure the size of the prediction errors.
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)
print(f'Mean Squared Error for Fare Prediction: {mse:.2f}')
print(f'Root Mean Squared Error: ${rmse:.2f}')
print(f'Coefficients (Age, Pclass): {lr_model.coef_}')
print(f'Intercept: {lr_model.intercept_:.2f}')
print('RMSE is a typical error scale, not the exact average absolute
```

```
Mean Squared Error for Fare Prediction: 3364.92
Root Mean Squared Error: $58.01
Coefficients (Age, Pclass): [ -0.43695889 -38.29728935]
Intercept: 133.31
RMSE is a typical error scale, not the exact average absolute error.
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Convert the text values into numbers for the model.
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
features_cls = ['Age', 'Pclass', 'Sex']
X_cls = df[features_cls]
y_cls = df['Survived']
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42
)
print(f'Classification: {len(X_train_cls)} training rows, {len(X_test_
```

```
Classification: 571 training rows, 143 test rows
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# Train a classification model to predict survival (0 or 1).
```

```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_cls, y_train_cls)

# Check predictions against the actual survival labels.
y_pred_cls = log_model.predict(X_test_cls)
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f'Accuracy for Survival Prediction: {accuracy:.2%}')
print(f'Correct predictions: {(y_pred_cls == y_test_cls).sum()} of {l
```

```
Accuracy for Survival Prediction: 74.83%
Correct predictions: 107 of 143
```

Knowledge Check

1. What is the key difference between regression and classification?

Regression predicts a number, such as a passenger's fare. Classification predicts a category, such as whether a passenger survived (1) or did not survive (0). Both use examples with known answers during training, so both are supervised learning.

2. What do `lr_model.coef_` values represent?

The two coefficients match the input columns in this order: `Age` and `Pclass`. Each one shows how much the predicted fare changes when that feature increases by one unit, assuming the other feature stays the same. The model treats ticket class as a number for this simple exercise. The coefficients describe the fitted model; they do not prove that age or ticket class causes a change in fare.

3. Why use MSE for fare prediction and accuracy for survival prediction?

Fare is a number, so MSE measures how far the predicted prices are from the actual prices, with larger mistakes counting more. Survival is a category, so accuracy measures the fraction of correct survival predictions. Accuracy would not tell us whether a fare prediction missed the real fare by $1 or 100$. RMSE converts MSE back into dollars, which makes the error easier to understand.